



Programming Fundamentals

By

Dr Suhail Asghar

Dawood University Of Engineering &
Technology Karachi

Pointers

Pointers

- A pointer is a variable whose value is the address of a memory location
- With pointers many memory locations can be referenced
- Extensively used in C++ for three main purposes:
 - To allocate new objects on the heap
 - To pass functions to other functions
 - To iterate over elements
- Modern C++ provides:
 - Smart pointers for allocating objects
 - Iterators for traversing data structures
 - Lambda expressions for passing functions

Pointer Variable Declaration

- **datatype *
variable_name ;**
 - * indicates variable is pointer**int *iPtr;**

iPtr is
pointer to data of type
integer

- **char *cPtr;**
cPtr is a pointer to
data of type character
- **double *dPtr;**
dPtr is a pointer to
data of type double

- Multiple **int** pointers
require multiple
asterisks

int *myPtr1, *myPtr2;

- **void * vPtr** declare
pointer to any data
type

Pointer initialization

- Initialized to **0**, **NULL**, or address
 - **0** or **NULL** points to nothing
- ```
int* p = nullptr; //
 declar & initialize
 pointer (doesn't store
 a random address)
//int variable
int i = 5;
p = &i; // assign
 pointer to address of
 object
int j = *p; //
 dereference p to
 retrieve value
```

# Pointer Operation

- **\*** (indirection/dereferencing operator)
  - Returns synonym for object its pointer operand points to
  - **\*yPtr** returns **y** (because **yPtr** points to **y**) .
  - dereferenced pointer is lvalue
    - \*yptr = 9;            // assigns 9 to y**
- **&** and **\*** are inverses of each other

# Pointer Operation

```
#include <iostream>
using std::cout;
using std::endl;
int main()
{
 int a; // a is an integer
 int *aPtr; // aPtr is a
 // pointer to an integer
 a = 7;
 aPtr = &a; // aPtr assigned
 // address of a
 cout << "The address of a is "
 << &a
 << "\nThe value of aPtr is "
 << aPtr;
```

```
 cout << "\n\nThe value of a is "
 << a
 << "\nThe value of *aPtr
 is " << *aPtr;
 cout << "\n\nShowing that * and &
 are inverses of "
 << "each other.\n&*aPtr =
 " << &*aPtr
 << "\n*&aPtr = " << *&aPtr
 << endl;
 return 0; // indicates successful
 // termination
} // end main
```

# const with pointers

```
int main()
{
 int x, y;
 int * const ptr = &x;
 *ptr = 7;
 ptr = &y; // error: ptr
 is const; cannot assign
 new address
 return 0;
}
```

- Attempting to modify a constant pointer to non-constant data.
- ptr is a constant pointer to an integer that can be modified through ptr, but ptr always points to the same memory location.



# Attempting to modify a constant pointer to constant data

```
#include <iostream>
using std::cout;
using std::endl;
int main() {
 int x = 5, y;
 const int *const ptr = &x;
 cout << *ptr << endl;
 *ptr = 7; // error: *ptr is
 const; cannot assign new value
 ptr = &y; // error: ptr is
 const; cannot assign new
 address
 return 0; }
```

- ptr is a constant pointer to a constant integer.
- ptr always points to the same location; the integer at that location cannot be modified.

# CALLING a function

- 3 ways to pass arguments to function
  - **Pass-by-value**
  - **Pass-by-reference with reference arguments**
  - **Pass-by-reference with pointer arguments**
- `return` can return one value from function
- **Arguments passed to function using reference arguments**
  - Modify original values of arguments
  - More than one value “returned”
- **Pass-by-reference with pointer arguments**
  - Simulate pass-by-reference
    - Use pointers and indirection operator
  - Pass address of argument using **&** operator
  - Arrays not passed with **&** because array name already pointer
  - **\*** operator used as alias/nickname for variable inside of function

# 1. Call by Value

- **How it works**
- A **copy** of the actual argument is passed to the function.
- Changes made inside the function **DO NOT affect** the original variable.

- **When to use**
- The function does not modify the original variable.
- Small data types (int, float, char) → copy is cheap.
- Safety: caller's data

```
void fun(int x) {
 x = 10; // modifies
 only local copy
}

int main() {
 int a = 5;
 fun(a);
 cout << a; // still 5
}
```

## 2. Call by Reference (int&)

- **How it works**

- Function gets a direct **alias** (another name) for the original variable.
- Any change made inside the function **WILL affect** the original variable.

- **When to use**

- The function modifies the caller's variable.
- When passing **large objects** (like vectors, strings, classes) to avoid copying.
- When returning multiple outputs from a function.

```
void fun(int &x)
{
 x = 10;
}
```

```
int main() {
 int a = 5;
 fun(a);
 cout << a;
 // now 10
}
```

```
void
compute(int a,
int b, int
&sum, int
&prod) {
 sum = a +
b;
 prod = a *
b;
}
```

```
int x = 4, y =
5;

int s, p;
```

# 3. Call by Pointer (int\*)

- **How it works**
- You pass the **address** of a variable.
- Function uses \*pointer to access/modify the original variable.
- Changes **affect the original variable**.
- **When to use**
- When you need to pass **null** (optional parameters).
- When working with **dynamic memory**, arrays.
- When interfacing with **C libraries** (C uses pointers, not references).
- When you want to pass **multiple values** via pointers.

```
void fun(int *x) {
 *x = 10;
}

int main() {
 int a = 5;
 fun(&a);
 cout << a; // now 10
}
```

# Parameter Passing: value vs reference vs pointer

## 1. Pass by value (copy)

```
void increment_by_value(int
n) {
 n = n + 1; // only local
 copy changes}
```

## 2. Pass by reference (modifies caller)

```
void increment_by_ref(int
&n) {
 n = n + 1; // original
 variable modified}
```

## 3. Pass by pointer (C-style)

Example showing all three:

```
#include <iostream>
using namespace std;
void pv(int n) { n++; }
void pr(int &n) { n++; }
void pp(int *n) { if(n) (*n)++; }
int main() {
 int a = 5;
 pv(a); cout << "after pv: " <<
a << '\n'; // 5
 pr(a); cout << "after pr: " <<
a << '\n'; // 6
 pp(&a); cout << "after pp: " <<
a << '\n'; // 7
}
```

Pass number by value; result  
returned by cubeByValue

```
#include <iostream>
using namespace std;

// cubeByValue returns
// the cube of the parameter
int cubeByValue(int n)
{
 n = n * n * n; //
 compute cube
 return n; //
 return result
}

int main() {
 int number = 5;
 cout << "The original
value of number is " <<
number;
 // pass number by
 value to cubeByValue
 number =
 cubeByValue(number);
 cout << "\nThe new
value of number is " <<
number << endl;
 return 0;
}
```

# Calling a function by reference

```
#include <iostream>
using namespace std;

// calculate cube of
*nPtr; modifies variable
number in main
void cubeByReference(int
*nPtr)
{
 *nPtr = *nPtr * *nPtr
 * *nPtr; // cube the
value
}
```

```
int main() {
 int number = 5;
 cout << "The original
value of number is " <<
number;
 // pass address of
number to cubeByReference

cubeByReference(&number);

 cout << "\nThe new
value of number is " <<
number << endl;

 return 0; } //
indicates successful
```



# Conversion (Lower to Upper Case)

```
#include <iostream>
#include <<cctype>
using namespace std;
// convert string to uppercase
letters
void convertToUppercase(char
*sPtr) {
 while (*sPtr != '\0') { //
current character is not '\0'
 if (islower(*sPtr))
// if character is lowercase,
 *sPtr =
toupper(*sPtr); // convert to
uppercase
 ++sPtr; // move to next
character in string
 } }
```

```
int main() {
 char phrase[] = "characters
and $32.98";
 cout << "The phrase before
conversion is: " << phrase;
 convertToUppercase(phrase);
 cout << "\nThe phrase after
conversion is: "
 << phrase << endl;
 return 0; // indicates
successful termination
}
```

# sizeof Function

- **sizeof**

- Unary operator returns size of operand in bytes
- For arrays, **sizeof** returns

( size of 1 element ) \*  
( number of elements )

- If **sizeof( int ) = 4**, then

```
int myArray[10];
cout
 << sizeof(myArray);
```

will print 40

- **sizeof** can be used with

- Variable names
- Type names
- Constant values

# sizeof Function

```
// Fig. 5.16: fig05_16.cpp
// Sizeof operator when used on an
// array name
// returns the number of bytes in
// the array.
#include <iostream>
using namespace std;
// return size of ptr
size_t getSize(double
*ptr)
{
 return sizeof(ptr);
}
```

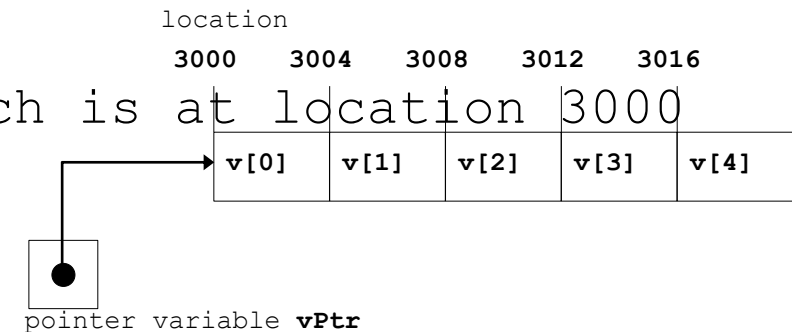
```
int main() {
 double array[20];
 cout << "The number
of bytes in the array is
"
 <<
sizeof(array);
 cout << "\nThe number
of bytes returned by
getSize is "
 <<
getSize(array) << endl;
 return 0; //
indicates successful
termination
```

# Pointer Expressions and Pointer Arithmetic

- Pointer arithmetic
  - Increment/decrement pointer (**++** or **--**)
  - Add/subtract an integer to/from a pointer( **+** or **+=** , **-** or **-=**)
  - Pointers may be subtracted from each other
  - Pointer arithmetic meaningless unless performed on pointer to array

- 5 elements **int** array on a machine using 4 byte  
**ints**

- **vPtr** points to first element **v[0]**, which is at location 3000  
**vPtr = 3000**
- **vPtr += 2;** sets **vPtr** to 3008  
**vPtr** points to **v[2]**



# Pointer Expressions and Pointer Arithmetic

- Subtracting pointers

- Returns number of elements between two addresses

```
vPtr2 = v[2]; vPtr = v[0];
vPtr2 - vPtr == 2
```

- Pointer assignment

- Pointer can be assigned to another pointer if both of same type
  - If not same type, cast operator must be used

- Pointer comparison

- Use equality and relational operators
  - Comparisons meaningless unless pointers point to members of same array
  - Compare addresses stored in pointers
  - Example: could show that one pointer points to higher numbered element of array than other pointer

# Relationship Between Pointers and Arrays

- Arrays and pointers closely related
  - Array name like constant pointer
  - Pointers can do array subscripting operations
- Accessing array elements with pointers
  - Element **b[n]** can be accessed by **\*( bPtr + n )**
    - Called pointer/offset notation
  - Addresses
    - **&b[3]** same as **bPtr + 3**
  - Array name can be treated as pointer
    - **b[3]** same as **\*( b + 3 )**
  - Pointers can be subscripted (pointer/subscript notation)
    - **bPtr[3]** same as **b[ 3 ]**

# Relationship Between Pointers and Arrays

```
#include <iostream>
using std::cout;
using std::endl;
int main() {
 int b[] = { 10, 20, 30, 40
};
 int *bPtr = b; // set bPtr
to point to array b
 // output array b using
array subscript notation
 cout << "Array b printed
with:\n"
 << "Array subscript
notation\n";
 for (int i = 0; i < 4; i++
)
 cout << "b[" << i << "]
= " << b[i] << '\n';
 // output array b using the
array name and
```

```
 for (int offset1 = 0; offset1
< 4; offset1++)
 cout << "* (b + " <<
offset1 << ") = "
 << *(b + offset1)
<< '\n';
 // output array b using bPtr
and array subscript notation
 cout << "\nPointer subscript
notation\n";
 for (int j = 0; j < 4; j++
)
 cout << "bPtr[" << j <<
"] = " << bPtr[j] << '\n';
 cout << "\nPointer/offset
notation\n";
 // output array b using bPtr
and pointer/offset notation
 for (int offset2 = 0;
offset2 < 4; offset2++)
 cout << "* (bPtr + " <<
```

# Relationship Between Pointers and Arrays (output)

Array b printed with:

Array subscript notation

b[0] = 10

b[1] = 20

b[2] = 30

b[3] = 40

Pointer/offset notation where the pointer is the array name

\* (b + 0) = 10

\* (b + 1) = 20

\* (b + 2) = 30

\* (b + 3) = 40

Pointer subscript notation

bPtr[0] = 10

bPtr[1] = 20

bPtr[2] = 30

bPtr[3] = 40

Pointer/offset notation

\* (bPtr + 0) = 10

\* (bPtr + 1) = 20

\* (bPtr + 2) = 30

\* (bPtr + 3) = 40



# Arrays of Pointers

- Arrays can contain pointers
  - Commonly used to store array of strings

```
char *suit[4] = {"Hearts", "Diamonds",
 "Clubs", "Spades" };
```
  - Each element of **suit** points to **char \*** (a string)
  - Array does not store strings, only pointers to strings
  - **suit** array has fixed size, but strings can be of any size

